Memoria
Sistemas Informáticos II P4

Rubén Juan Jiménez Blázquez y José Antonio Gómez Martínez

Ejercicio 1:

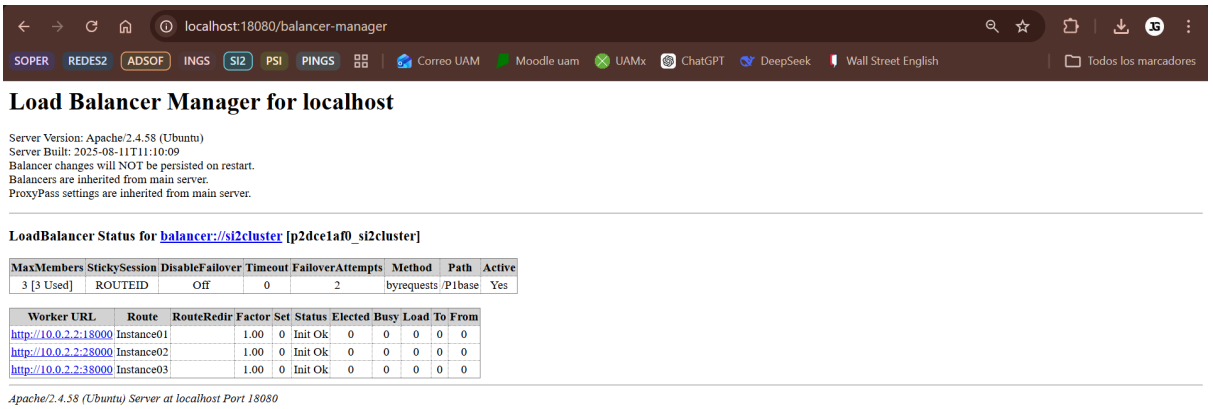
Captura del sudo systemctl status apache2:

```
(venv) si2@si2-ubuntu-vm-1:~/SI2-Practica_4/P1-base$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-04-27 11:10:32 CEST; 16min ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 80491 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 80494 (apache2)
      Tasks: 55 (limit: 1670)
     Memory: 5.9M (peak: 6.1M)
        CPU: 489ms
     CGroup: /system.slice/apache2.service
             ├─80494 /usr/sbin/apache2 -k start
             ├─80495 /usr/sbin/apache2 -k start
             └─80497 /usr/sbin/apache2 -k start

abr 27 11:10:32 si2-ubuntu-vm-1 systemd[1]: Starting apache2.service - The Apache HTTP Server...
abr 27 11:10:32 si2-ubuntu-vm-1 apachectl[80493]: AH00558: apache2: Could not reliably determine the server's fully qualified domain>
abr 27 11:10:32 si2-ubuntu-vm-1 systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-17/17 (END)
```

Captura del sudo apachectl configtest:

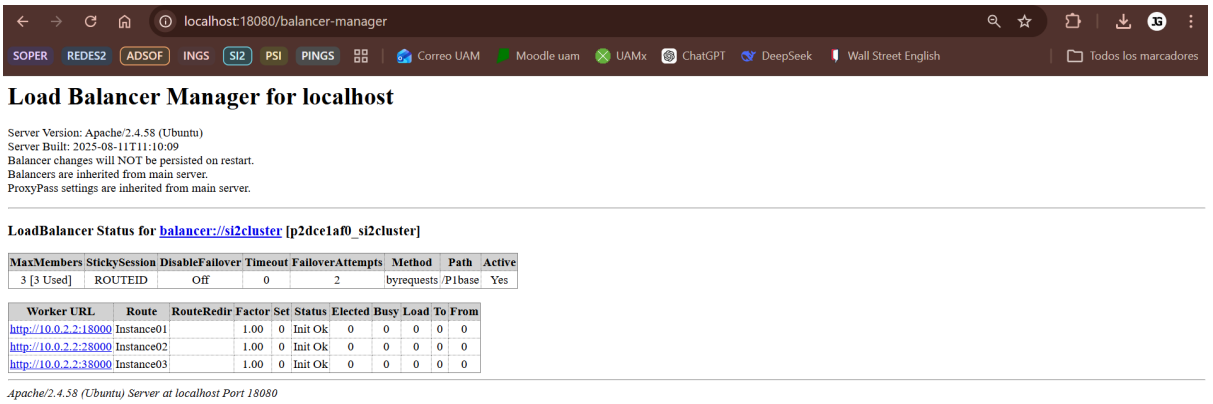
```
(venv) si2@si2-ubuntu-vm-1:~/SI2-Practica_4/P1-base$ sudo apachectl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' direct
ive globally to suppress this message
Syntax OK
```

Captura de la consola del balanceador:



Ejercicio 2:

Balanceador de carga inicial:

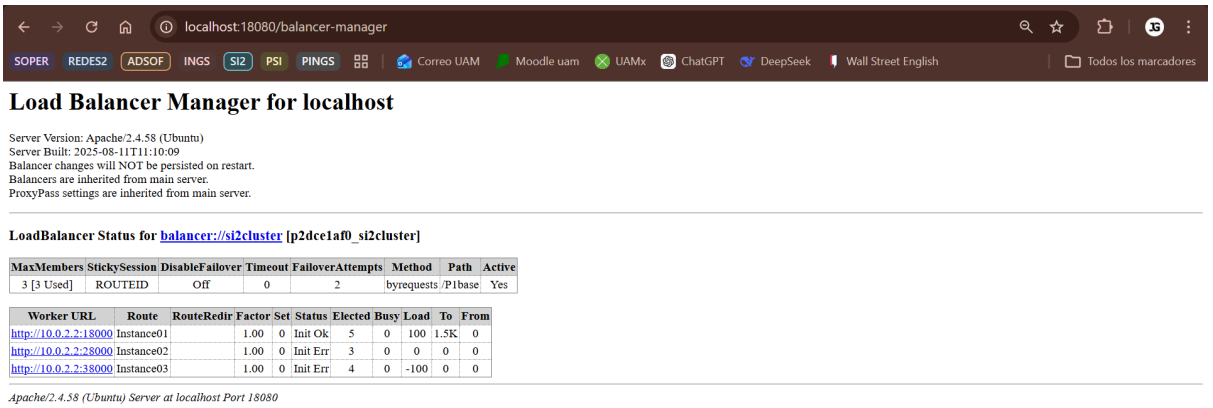


Captura 1: El fallo inicial (Simulación de caída)

Esto demuestra que, al estar los Gunicorns de VM2 y VM3 apagados, el balanceador no siempre encuentra un camino válido y el usuario recibe un error.

Captura 2: Estado de error en el Balancer Manager

Las filas de Instance02 e Instance03 muestran un estado de Init Err o. La columna "Elected" tiene números, indicando que Apache ha intentado usarlas.



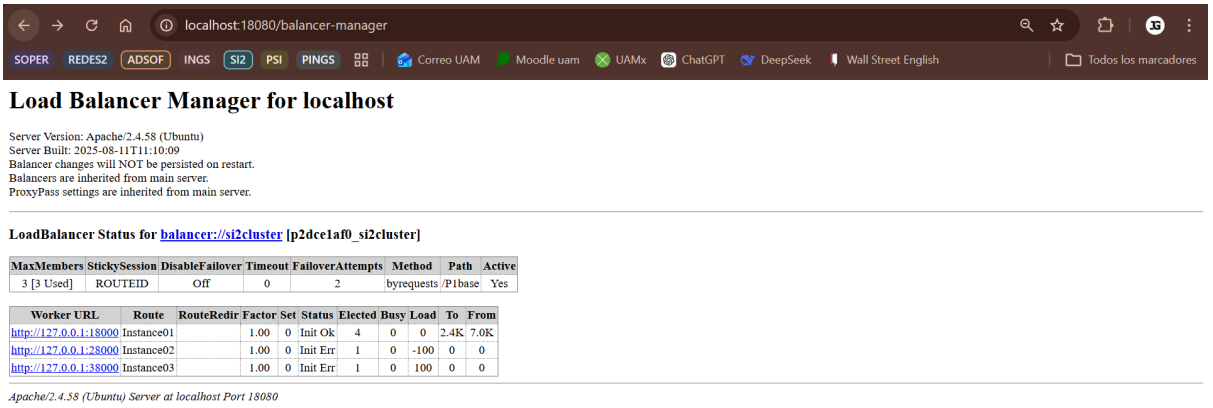
Captura 3: Tolerancia a fallos con curl (Éxito parcial)

Probamos que el balanceador, tras detectar los fallos, es capaz de redirigir el tráfico al único nodo vivo (VM1) y servir la página correctamente.

```
si2@si2-ubuntu-vm-1:~$ curl -L http://10.0.2.15/P1base/visaApp/
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Aportar Informacion Tarjeta (visaSite)</title>
</head>
<body>
    <h1>Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)</h1>

    <form method="post">
        <input type="hidden" name="csrfmiddlewaretoken" value="j4KIxF8EWBETlI6AG5432aLxZGs86HjhVgIV3EgkcUO1e4XFphf9tm88nEt8xdUD">
        <p>
    <label for="id_numero">Número de Tarjeta:</label>
    <input type="text" name="numero" required id="id_numero">


  </p>


  <p>
    <label for="id_nombre">Nombre:</label>
    <input type="text" name="nombre" required id="id_nombre">


  </p>


  <p>
    <label for="id_fechaCaducidad">Fecha de Caducidad:</label>
    <input type="text" name="fechaCaducidad" required id="id_fechaCaducidad">


  </p>


  <p>
    <label for="id_codigoAutorizacion">Código de Autorización:</label>
    <input type="text" name="codigoAutorizacion" required id="id_codigoAutorizacion">



  </p>
        <button type="submit">Enviar Información Tarjeta</button>
    </form>
</body>
</html>
```

Captura 4: Balancer Manager tras la recuperación

Esto confirma visualmente que todo el tráfico del clúster está siendo absorbido por el único nodo disponible, manteniendo la disponibilidad del servicio.



Respuesta al Ejercicio 6 (Comportamiento del balanceador)

¿En qué nodos se están ejecutando las peticiones?: Al inicio de la prueba, con los nodos 02 y 03 apagados, las peticiones fallan (error 502/503). Una vez que el balanceador detecta el fallo, todas las peticiones con éxito se ejecutan exclusivamente en la VM1. Esto se

comprueba en la captura del Balancer Manager, donde la Instance01 es la única con estado "Ok" y un número creciente de peticiones "Elected".

¿En qué momento detecta Apache que hay dos nodos apagados?: Apache no realiza una detección preventiva. Detecta el fallo bajo demanda, es decir, en el momento en que un usuario intenta acceder y la conexión con el nodo (Gunicorn) es rechazada. En ese instante, marca el nodo como Init Err.

Respuesta al Ejercicio 7 (Diferencia Navegador vs Curl)

¿Los resultados son diferentes?: Sí, el comportamiento varía significativamente entre ambas herramientas.

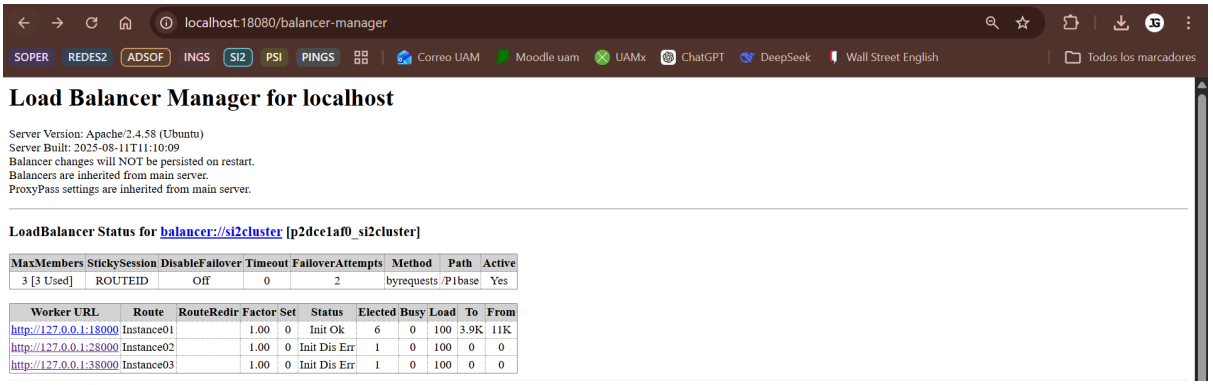
¿Por qué?: El comando curl es una herramienta sin estado que no gestiona cookies de sesión por defecto. Por ello, en cada petición Apache intenta balancear la carga de nuevo, chocando repetidamente con los nodos caídos y devolviendo errores 502. En cambio, el navegador soporta Sticky Sessions mediante la cookie ROUTEID. Una vez que el navegador consigue conectar con el nodo vivo (VM1), almacena la cookie y Apache le asigna siempre ese nodo en las siguientes peticiones, evitando así los errores y ofreciendo una navegación estable.

Ejercicio 3:

En este ejercicio se ha configurado el clúster para utilizar una base de datos PostgreSQL compartida ubicada en la VM1. Además, se ha modificado el modelo de datos para registrar qué instancia del clúster procesa cada pago, demostrando así la integración entre el balanceador de carga y la persistencia de datos.

Paso 1: Configuración del Balanceador (Captura 1)

Para garantizar que la prueba sea concluyente, se ha accedido a la consola del Balancer Manager (http://localhost:18080/balancer-manager). Se han deshabilitado manualmente las instancias 02 y 03 (estado Dis), forzando así a Apache a redirigir todo el tráfico entrante exclusivamente a la Instance01 (VM1), la cual permanece en estado Ok.



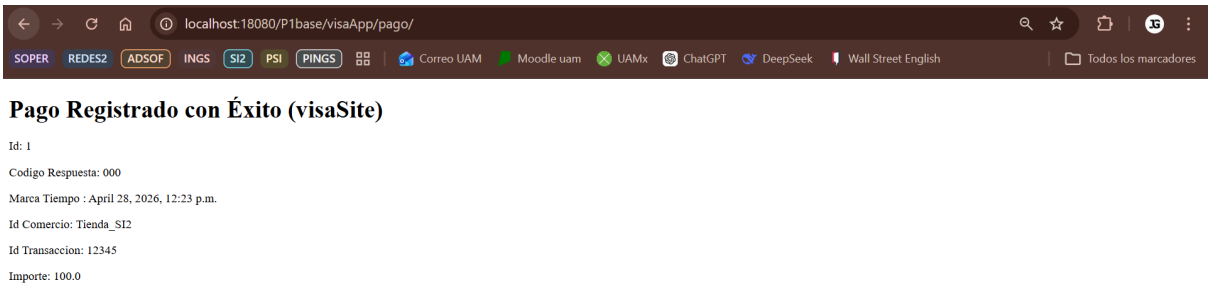
MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:18000	Instance01		1.00	0	Init Ok	6	0	100	3.9K	11K
http://127.0.0.1:28000	Instance02		1.00	0	Init Dis Err	1	0	100	0	0
http://127.0.0.1:38000	Instance03		1.00	0	Init Dis Err	1	0	100	0	0

Paso 2: Registro de Pago a través del Clúster (Captura 2)

Una vez forzado el tráfico a la VM1, se ha procedido a realizar un pago completo desde el navegador a través de la URL del balanceador. Se han utilizado los datos de una tarjeta

válida previamente cargada en la base de datos PostgreSQL mediante el script de población (populate.py). El sistema ha procesado la solicitud correctamente, devolviendo la pantalla de "Pago Registrado con Éxito". Esta captura confirma que la aplicación es capaz de realizar transacciones completas operando detrás de un balanceador de carga.



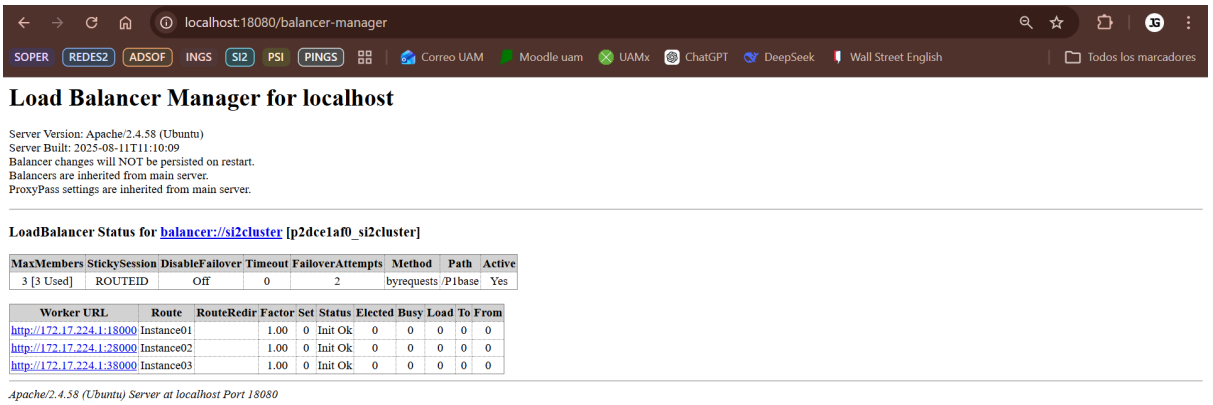
Paso 3: Verificación de Persistencia e Instancia (Captura 3)

Finalmente, se ha validado la información almacenada en PostgreSQL accediendo a la shell de Django en la VM1. Al consultar el último registro de la tabla Pago, se comprueba que los datos coinciden con la transacción realizada (ID 1, Importe 100.0). Lo más relevante es que el campo INSTANCIA muestra el valor Instance01, lo que certifica que el servidor ha identificado correctamente la ruta asignada por Apache a través de la cookie ROUTEID y ha persistido dicha información en la base de datos centralizada.

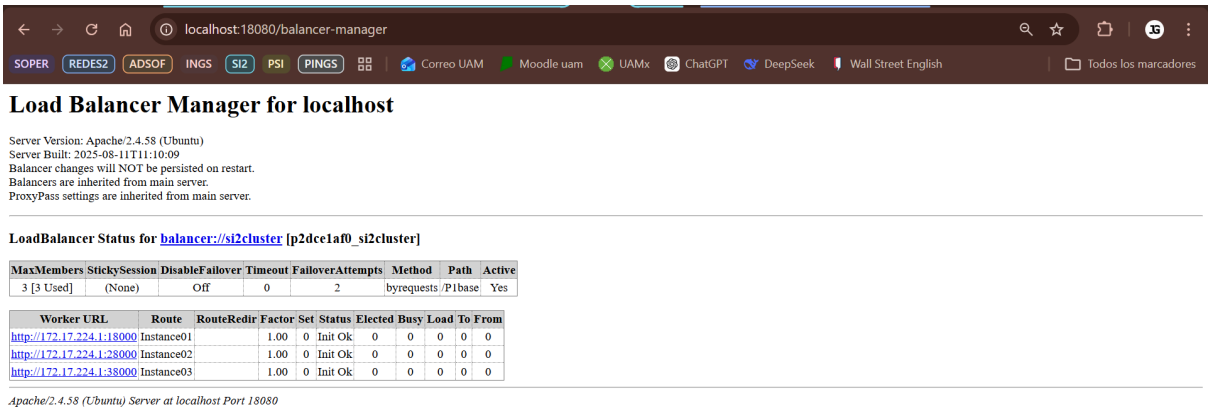
```
(venv) si2@si2-ubuntu-vm-1:~/SI2-Practica_4/P1-base$ python manage.py shell -c "from visaApp.models import Pago; p = Pago.objects.las
t(); print(f'\n--- DATOS DEL PAGO ---\nID: {p.id}\nComercio: {p.idComercio}\nImporte: {p.importe}\nINSTANCIA: {p.instancia}\n--------
----------')"
8 objects imported automatically (use -v 2 for details).


--- DATOS DEL PAGO ---
ID: 1
Comercio: Tienda_SI2
Importe: 100.0
INSTANCIA: .Instance01
----------
```

Ejercicio 4:

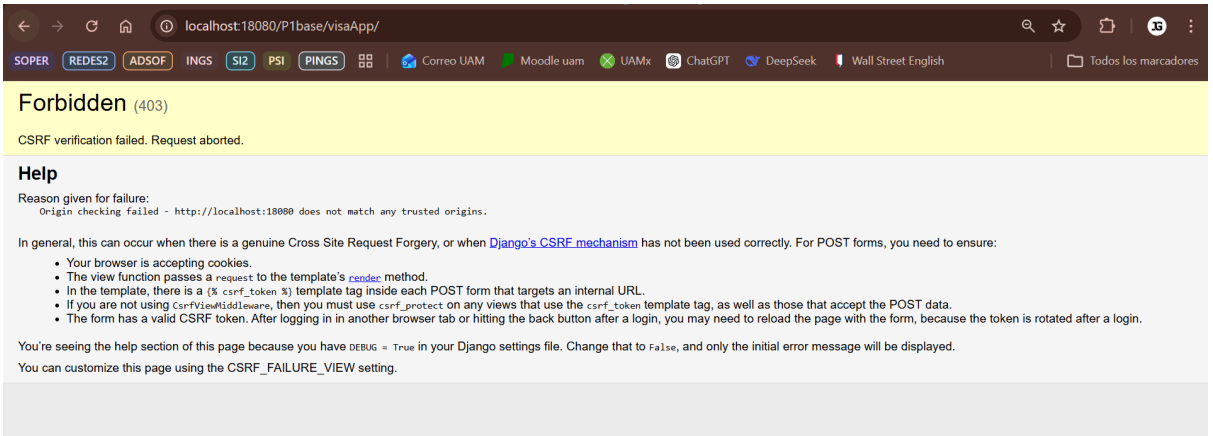
Captura del Balance Manager inicial:

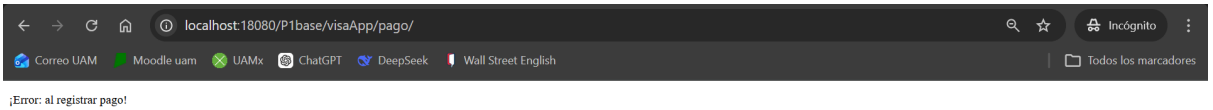


Captura del Balance Manager sin StickySession:



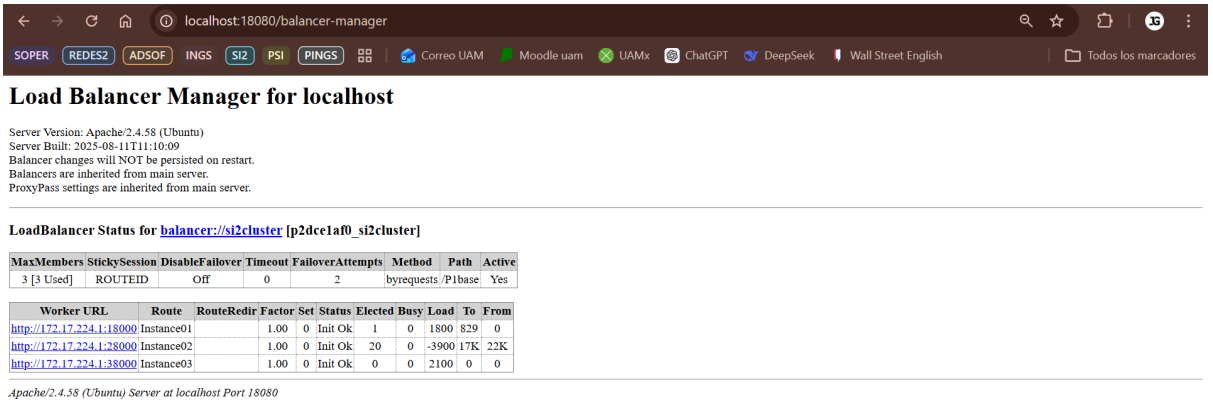
"Al suprimir la afinidad de sesión, se produce un error de verificación CSRF (como se muestra en la captura). Esto ocurre porque el balanceador de carga redirige la solicitud POST a una instancia diferente de la que sirvió el formulario original. Dado que el token de seguridad CSRF es local a la sesión de la instancia que lo generó, la nueva instancia no puede validarlo y aborta la operación por seguridad. Este fallo demuestra que la afinidad de sesión es imprescindible para que aplicaciones basadas en frameworks como Django puedan procesar formularios de forma segura en un clúster."



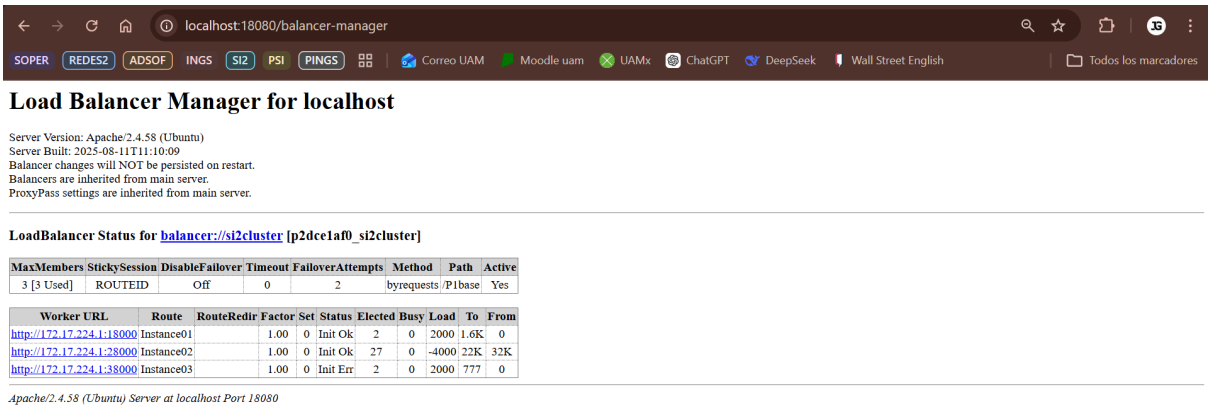


Ejercicio 5:

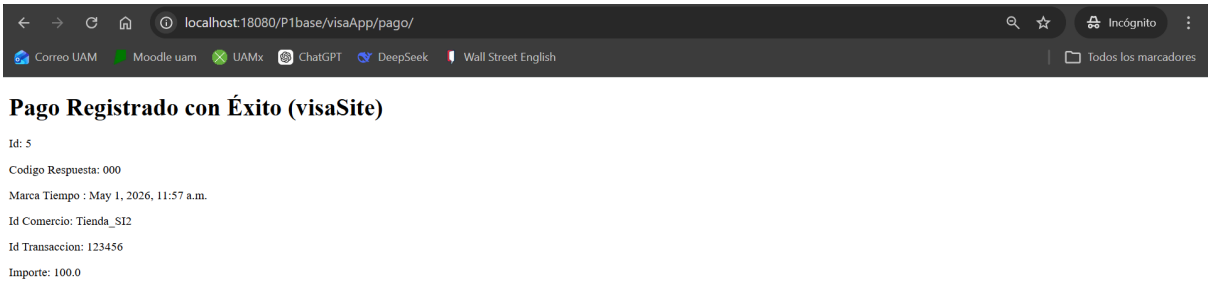
Esta es la captura de como esta el balanceador al principio y como se puede ver las tres instancias están vivas (**Init Ok**) y la **Instance03** es la que menos carga tiene (0 elecciones):



Ahora vamos a simular el fallo crítico en la MV3 (la que corresponde a la Instance03 en el puerto 38000), ejecutando el comando del enunciado: sudo poweroff



Como se observa en la captura, al caer la Instance03, el balanceador redirige el tráfico hacia las dos instancias restantes (Instance01 e Instance02), lla carga se reparte equilibradamente entre los dos nodos que permanecen operativos, garantizando la continuidad del servicio.



Al realizar una nueva petición de pago desde el cliente, el balanceador redirige la carga automáticamente hacia el nodo disponible (Instance01), permitiendo que la transacción se complete satisfactoriamente en la base de datos centralizada.

Ejercicio 6:

Primero comprobamos el estado inicial del balanceador accediendo a /balancer-manager y verificamos que las tres instancias están en "Init Ok".

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://si2cluster [pc09dfdc3_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:8001	Instance01		1.00	0	Init Ok	1	0	0	198	15
http://10.0.2.2:28000	Instance02		1.00	0	Init Ok	2	0	-300	1.6K	3.4K
http://10.0.2.2:38000	Instance03		1.00	0	Init Ok	0	0	300	0	0

Apache/2.4.58 (Ubuntu) Server at localhost Port 18080

Luego apagamos la máquina VM2 con sudo poweroff para simular una caída. Tras refrescar el balanceador varias veces, observamos que Instance02 pasa a estado de error.

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://si2cluster [pc09dfdc3_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:8001	Instance01		1.00	0	Init Ok	1	0	200	198	15
http://10.0.2.2:28000	Instance02		1.00	0	Init Err	3	0	-500	1.6K	3.4K
http://10.0.2.2:38000	Instance03		1.00	0	Init Ok	1	0	300	846	1.1K

Apache/2.4.58 (Ubuntu) Server at localhost Port 18080

Encendemos de nuevo VM2 desde VirtualBox y nos conectamos por SSH. Verificamos que gunicorn está funcionando ejecutando un curl local. Volvemos a refrescar /balancer-manager y vemos que Instance02 ha vuelto al estado "Init Ok".

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://si2cluster [pc09dfdc3_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:8001	Instance01		1.00	0	Init Ok	0	0	0	0	0
http://10.0.2.2:28000	Instance02		1.00	0	Init Ok	0	0	0	0	0
http://10.0.2.2:38000	Instance03		1.00	0	Init Ok	0	0	0	0	0

Apache/2.4.58 (Ubuntu) Server at localhost Port 18080

Después hacemos 3 pagos completando el formulario. Tras los pagos, verificamos nuevamente el balanceador y observamos que Instance02 ha aumentado su contador de peticiones ("Elected").

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://si2cluster [pc09dfdc3_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:8001	Instance01		1.00	0	Init Ok	4	0	1500	4.0K	2.3K
http://10.0.2.2:28000	Instance02		1.00	0	Init Ok	5	0	1200	4.7K	4.8K
http://10.0.2.2:38000	Instance03		1.00	0	Init Ok	18	0	-2700	18K	18K

Apache/2.4.58 (Ubuntu) Server at localhost Port 18080

Pago Registrado con Éxito (visaSite)

Id: 1

Codigo Respuesta: 000

Marca Tiempo : May 1, 2026, 7:16 p.m.

Id Comercio: IDC123

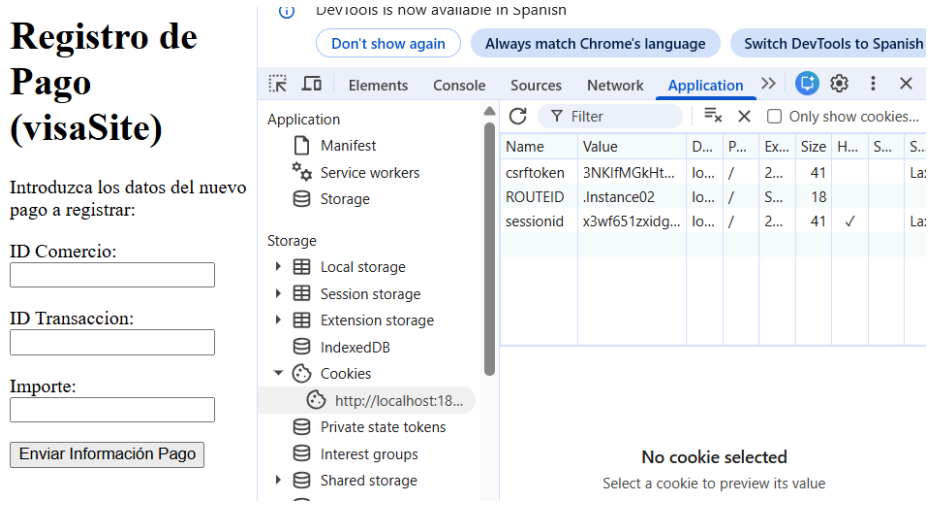
Id Transaccion: IDT123

Importe: 20.0

Instance02 es reactivada correctamente tras el reinicio y el balanceador empieza a enviarle peticiones nuevamente, demostrando que el failback funciona como se espera.

Ejercicio 7:

En esta prueba iniciamos una sesión de pago desde el navegador y completamos el formulario de tarjeta. Antes de enviar el formulario de pago, comprobamos en las cookies que el valor de ROUTEID era Instance02.



Con la sesión activa, apagamos la máquina correspondiente a Instance02 justo antes de enviar el pago.

```
(venv) si2@si2-ubuntu-vm-2:~/SI2-Practica_4/P1-base$ sudo poweroff

Broadcast message from root@si2-ubuntu-vm-2 on pts/1 (Fri 2026-05-01 19:40:1
1 CEST):

The system will power off now!

(venv) si2@si2-ubuntu-vm-2:~/SI2-Practica_4/P1-base$ Connection to localhost
 closed by remote host.
Connection to localhost closed.
```

Después completamos el formulario de pago y enviamos la petición. El balanceador redirige automáticamente la solicitud a Instance01, observándose el cambio en ROUTEID, por lo que el failover se produjo correctamente.

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

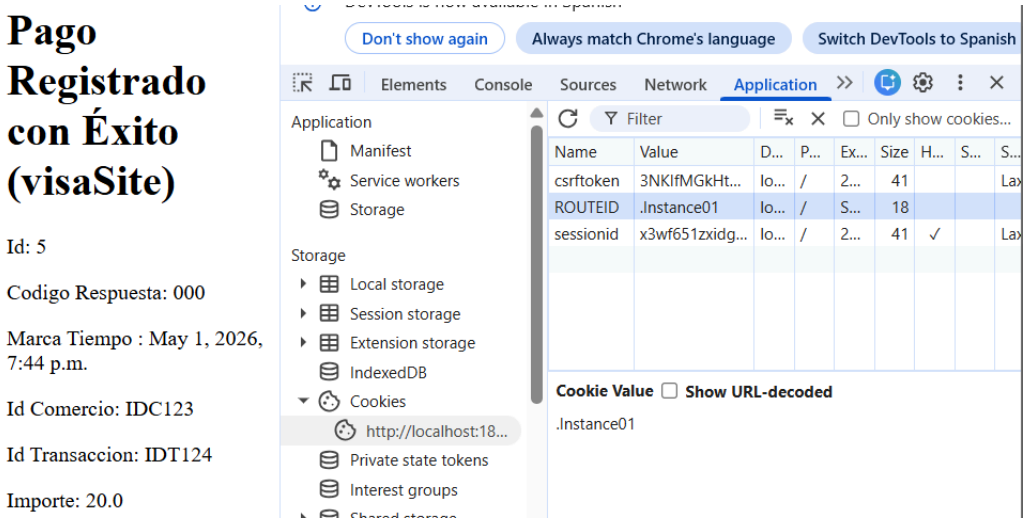
LoadBalancer Status for balancer://si2cluster [pc09dfdc3_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:8001	Instance01		1.00	0	Init Ok	17	0	500	15K	22K
http://10.0.2.2:28000	Instance02		1.00	0	Init Err	15	0	-100	12K	21K
http://10.0.2.2:38000	Instance03		1.00	0	Init Ok	18	0	-400	18K	18K

Apache/2.4.58 (Ubuntu) Server at localhost Port 18080

Finalmente, el pago se registra con éxito tras la redirección, verificando que el sistema mantiene la continuidad del servicio cuando cae la instancia que estaba procesando la sesión activa.



Ejercicio 8:

Para este ejercicio configuramos el plan de pruebas de JMeter a partir del fichero de la práctica anterior y lo adaptamos para ejecutar la aplicación P1-base contra el balanceador. Ajustamos el plan para trabajar con 1 hilo, 1000 repeticiones y sin tiempos de espera artificiales, y lanzamos la prueba sobre la URL del clúster.

Antes de ejecutar la prueba, limpiamos la tabla de pagos para no mezclar resultados anteriores con los nuevos datos. Durante la ejecución comprobamos el comportamiento del balanceador en la consola de Apache y revisamos el Aggregate Report al finalizar.

Al terminar la prueba, consultamos la base de datos y vimos que los pagos se registraron correctamente. La consulta devolvió que los 1000 registros quedaron asociados a .Instance03.

Etiqueta ↑	# Muestras	Mín	Máx	Rendimiento	% Error
Get Tarjeta Fr...	1000	12	1164	4,0/sec	0,00%
Post Pago	1000	45	3093	4,0/sec	0,00%
Post tarjeta	1000	66	588	4,0/sec	0,00%
Total	3000	12	3093	12,1/sec	0,00%

```
[{'instancia': '.Instance03', 'total': 1000}]
```

Sin embargo, en la consola de Apache observamos que el contador Elected quedó repartido de forma equilibrada entre las tres instancias:

Load Balancer Manager for localhost

Server Version: Apache/2.4.58 (Ubuntu)
Server Built: 2026-03-05T17:31:54
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://si2cluster [pc09dfdc3_si2cluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
3 [3 Used]	ROUTEID	Off	0	2	byrequests	/P1base	Yes

Worker URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From
http://127.0.0.1:8001	Instance01		1.00	0	Init Ok	4000	0	0	2.4M	2.3M
http://10.0.2.2:28000	Instance02		1.00	0	Init Ok	4000	0	0	2.4M	2.3M
http://10.0.2.2:38000	Instance03		1.00	0	Init Ok	4000	0	0	2.5M	2.4M

Apache/2.4.58 (Ubuntu) Server at localhost Port 18080

Se ejecutó exitosamente el test de carga con JMeter (1000 muestras, 1 thread) contra el balanceador. El test se completó sin errores (0% error rate), demostrando que el sistema distribuido procesa correctamente las transacciones bajo carga. La consulta a la base de datos verificó que los 1000 pagos se registraron correctamente, todos asociados a la instancia Instance03 (la que procesó la sesión debido al cookie ROUTEID de sticky session). Los contadores del balancer-manager mostraron actividad en las tres instancias, confirmando que el balanceador está operativo y enrutando tráfico hacia los diferentes nodos.